

# 基于 Benders 分解的 环形生产线平衡： 自顶向下和自底向上方法（算法设计）

郭鹏辉 博士生（Min 团队）

<https://guo.ph>

南京航空航天大学 经济与管理学院

智能决策与风险分析研究所



南京航空航天大学  
NANJING UNIVERSITY OF AERONAUTICS AND ASTRONAUTICS

经济与管理学院

2023年9月4日

# 目录

- 赛题简述和关键点
- Benders 分解
- **问题分解（自顶向下） 不拆分工序**
- **算法增强**
- 问题分解（自顶向下） 拆分工序
- 问题分解（自底向上） 复杂工序拓扑
- 算法流程
- 数值试验结果
- 算法优势
- 结论和展望

# 赛题简述和关键点

- 赛题简述

- 将若干 **工序** 分配给若干个 **工人** 和 **站位**，使得工序执行满足工艺规则约束、人员技能约束、站位机器约束、节拍均衡约束、重复入站和最大转圈数量约束。生成可执行（可行解）的工序到工人和站位的分配关系，并最小化赛题评价指标。（详见赛题文档）

- 关键点及难点

- **数学模型规模巨大**：若直接用混合整数规划（Mixed-Integer Programming, MIP）建模，数学模型的规模与  $|\text{工序}| \times |\text{工人}| \times |\text{站位}| \times |\text{圈数}|$  成比例。公开算例的规模可以达到几十万级。且调度 / 排序问题的线性松弛较弱，而 MIP 依赖求解线性松弛问题找到可行解，使得难以找到可行解。
- **环形生产线的建模**：**环形生产线平衡（Circular Assembly Line Balancing）** 是本次比赛首次提出的新的生产线平衡问题（Assembly Line Balancing Problem, ALBP），无针对“工件转圈”的成熟建模方式。而转圈数（公开算例中最大为8）又会极大地影响数学模型的规模。
- **重复入站约束建模**：重复入站是赛题的关键约束，与“环形生产线”有密切联系，无成熟建模方式。
- **工序拆分的建模**：工序拆分是赛题的关键约束，部分算例在不允许工序拆分时无可行解。如何对工序拆分建模（如目标函数和工序先后关系约束等），并保证所设计的算法具有通用性是赛题的又一关键点。



# Benders 分解

## 方法选择动机

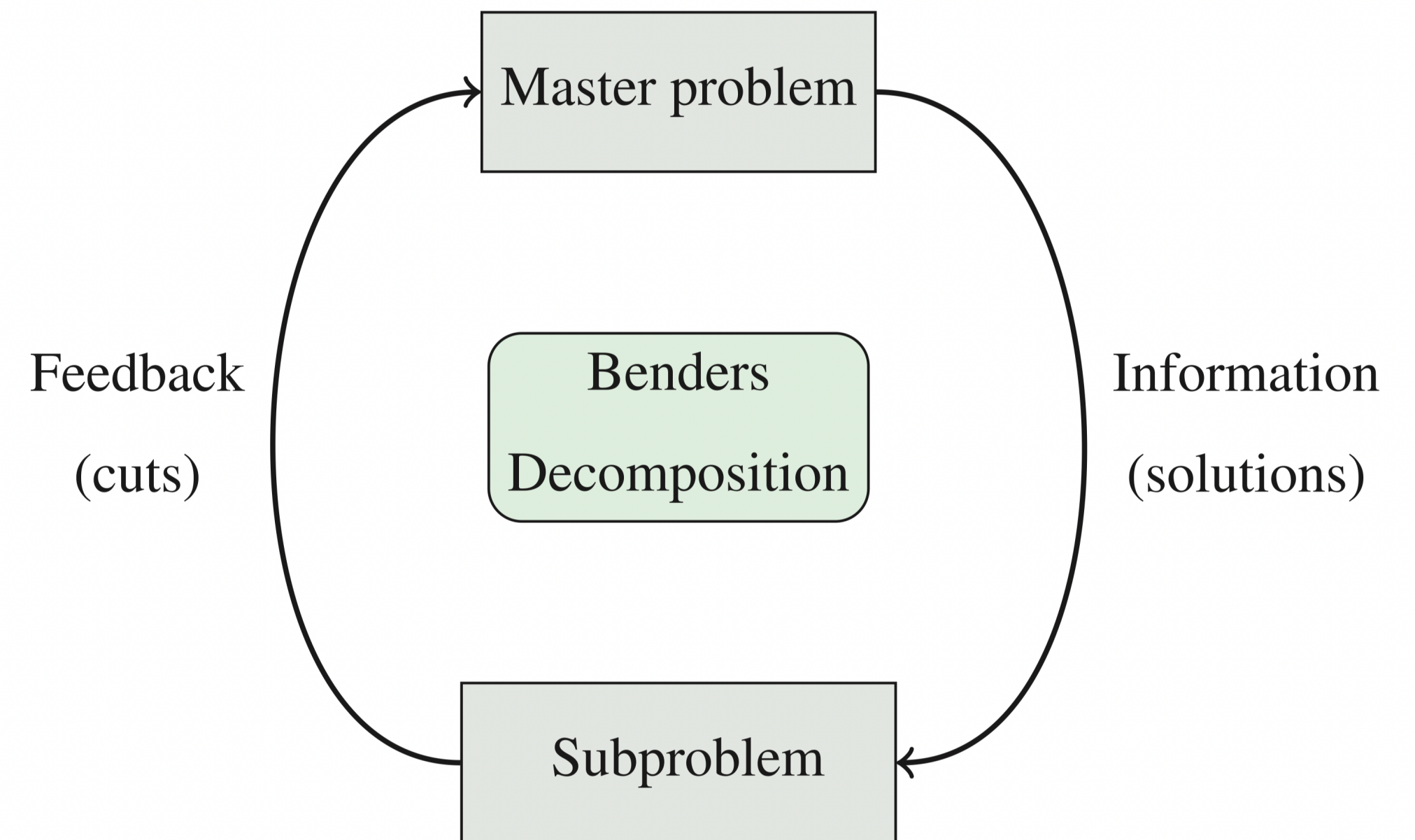
- 整体的 MIP 难以求解赛题（甚至无法给出可行解），尤其是在比赛限制使用开源求解器的条件下。因此考虑**对整体的 MIP 模型进行拆分，从而降低单次求解的问题的规模**，以解决**关键点及难点之“数学模型规模巨大”**。
- 1962年 Benders, J. F. 提出的 **Benders 分解（Benders Decomposition, BD）** 正是基于这一思想的确定性求解框架。
- 但经典的 Benders 分解能够求解的问题类型十分局限，具体来说，要求原始问题可以拆分成一个 MIP 和一个线性规划（Linear Programming, LP）问题。**以赛题为例，显然无法仅用连续变量和线性约束刻画。**
- 2006年 Codato & Fischetti 以及2003年 Hooker & Ottosson 分别提出的**组合 Benders 分解（Combinatorial Benders Decomposition, CBD）**和**基于逻辑的 Benders 分解（Logic-based Benders Decomposition, LBBD）**在一定程度上解决了 BD 的局限性。
- CBD 的核心 - **组合 Benders 割（Combinatorial Benders Cut）** 和 LBBD 的核心 - **子问题可以为任何形式的优化问题**，适用于赛题含有大量组合（0-1）变量以及包含排序决策的问题特点。



# Benders 分解

## 理论概述

- **原理**：Benders 分解的基本思想是将一个大型的优化问题分解为两个更小、更易于处理的子问题。**主问题 (Master problem)** 处理一部分决策变量，而**子问题 (Subproblem)** 处理另一部分决策变量。
- **分解**：在 Benders 分解中，原问题被分解为一个主问题和一个或多个子问题。主问题通常是一个整数规划问题，而子问题是一个**线性规划问题 (BD)**、或**不可行性证明 (CBD)**、或其他形式的优化问题 (LBBD), 如约束规划 (Constraint Programming, CP)。
- **迭代**：Benders 分解通过迭代过程来解决问题。在每一次迭代中，先解决主问题，然后解决子问题。通过这种方式，我们可以逐步改善解的质量。
- **Benders割**：通过解决子问题，我们可以得到 **Benders 割 (Benders cuts)**，即线性不等式。这些切割被添加到主问题中，以改善其解的质量。Benders 割分为**最优性割 (Optimality cuts)** 和**可行性割 (Feasibility cuts)**。
- **收敛**：Benders 分解的迭代过程会在满足一定条件时停止，此时我们可以得到原问题的最优解。这个条件通常是主问题和子问题的目标函数值之差小于某个预定的阈值，或证明当前主问题解的质量无法进一步提升。

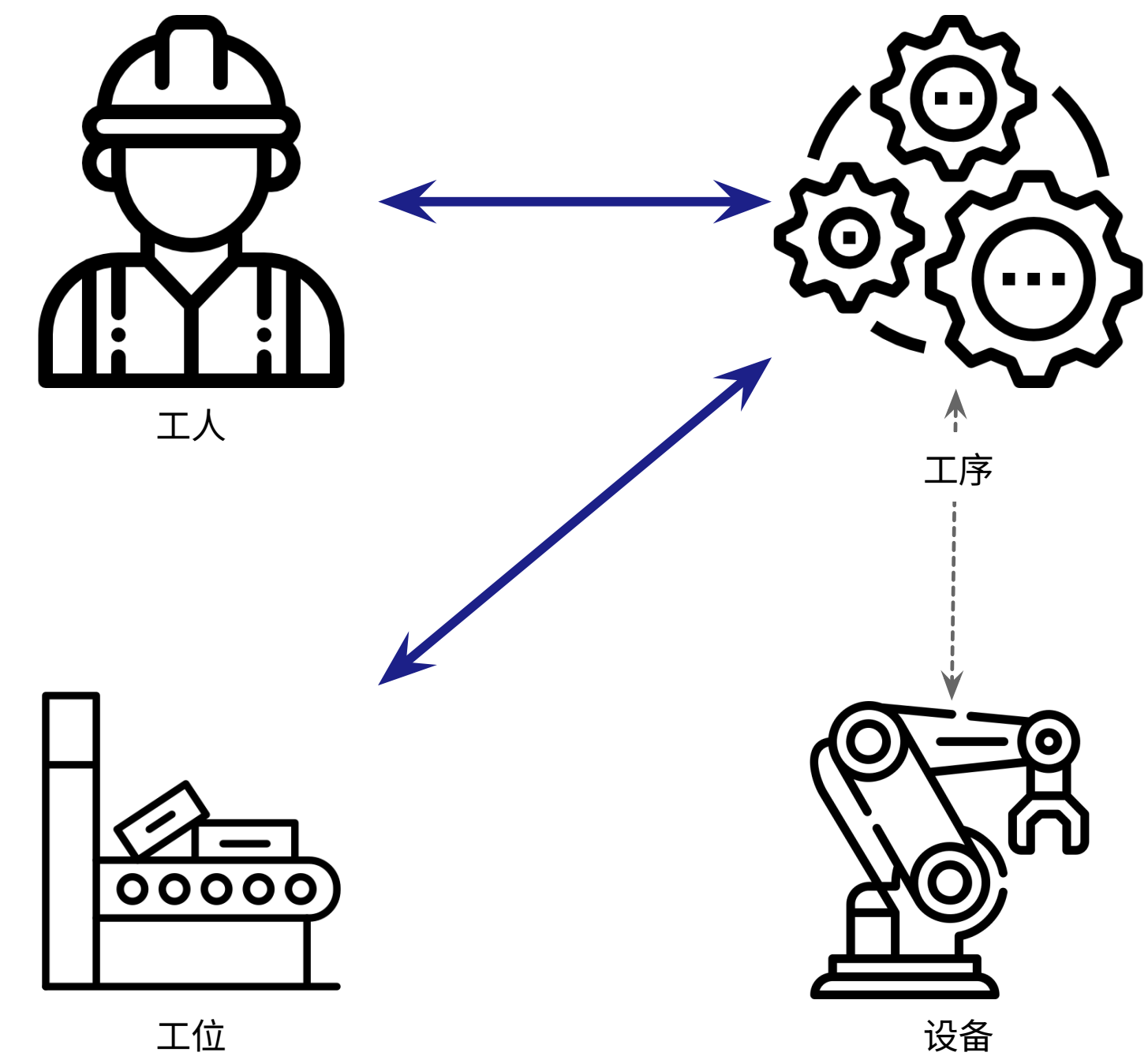


Benders 分解算法结构  
来源：参考文献 5

# Benders 分解

## 赛题分析

- 题目中包含四组分配关系：**工序到工人的分配**，**工序到工位的分配**，**工人到工位的分配**，和**设备到工位的分配**。
- 前三者任意两者确定，第三者也相应确定；设备到工位的分配隐含于工序到工位的分配。因此实质上只包含两组分配关系：**工序到工人的分配**和**工序到工位的分配**。
- **自顶向下分解**：适用于工序间依赖关系简单的算例
  - **主问题**：工序到工人的分配
  - **子问题**：工序到工位的分配
- **自底向上分解**：适用于工序间依赖关系复杂的算例
  - **主问题**：工序到工位的分配
  - **子问题**：工序到工人的分配



分配关系  
icons 来源: freepik.com

# 问题分解（自顶向下）不拆分工序

## 主问题：集合、变量和目标函数

- 主问题采用 **MIP** 建模，建模语言为 **Pyomo**，求解器为 **HiGHS**。
- 集合：**  $W$  表示工人的集合；  $P$  表示工序的集合
- 变量：**
  - 0-1 变量  $x_{wp}$  表示工人  $w$  是否分配给工序  $p$ ，  $x_{wp} = 1$ 表示是，反之否。
  - 连续变量  $y_w$  表示工人  $w$  的节拍，  $y_w = standardOperTime_p/efficiency_{wp} \cdot x_{wp}$ ，其中  $standardOperTime_p$  为工序  $p$  的标准工时，  $efficiency_{wp}$  为工人  $w$  做工序  $p$  的效率。
- 原始的评价包含二次项和分式等，形式较复杂，因此采用如下**代理目标函数**：
  - 最小化最大节拍：**  $Min. \max_{w \in W} y_w$ （易线性化，见源代码 UALB\_CB.py, Solver, make\_master\_problem）
  - 最小化平均节拍：**  $Min. \sum_{w \in W} y_w / |W|$
  - 最小化基尼系数：**  $Min. \sum_{w_1, w_2 \in W} |y_{w_1} - y_{w_2}|$ （易线性化，见源代码 UALB\_CB.py, Solver, make\_master\_problem）
- 0 目标函数：**  $Min. 0$ （将优化问题转化为可行性问题）
- 最终 **实际参与运算的是“0 目标函数”**，即将原始的优化问题转化为可行性问题。虽然“最小化最大节拍”在不显著增加计算时间的条件下可以实现较小的真实评价指标，但“0 目标函数”在更多的算例上找到了可行解。“最小化基尼系数”会显著增加运算时间。“最小化平均节拍”找到的可行解比“0 目标函数”少，实现的真实评价指标比“最小化最大节拍”大。



不拆分工序



# 问题分解（自顶向下） 不拆分工序

## 主问题：约束

- 每个工序分配给一个工人： $\sum_{w \in W} x_{wp} = 1$
- 每个工人至少有一个工序： $\sum_{p \in P} x_{wp} \geq 1$
- 工序技能和技能类要求： $\sum_{w \in W} x_{wp} \leq \max(skillCap_{wp}, categoryCap_{wp})$ ，其中  $skillCap_{wp}, categoryCap_{wp} \in \{0,1\}$  表示工人  $w$  是否具有工序  $p$  所需技能或技能类。
- 优先分配工序给具有相同技能的工人： $\sum_{w \in W_p} x_{wp} = 1$ ； $\sum_{w \in W - W_p} x_{wp} = 0$ ，其中  $W_p = \{w \in W \mid skillCapable_{wp} = 1\}$  表示具有工序  $p$  所需技能的工人集合。
- 固定某些工序给特定的工人： $x_{\bar{w}_p p} = 1, \forall p \in P; x_{wp} = 0, \forall w \in W - \{\bar{w}_p\}$ ，其中  $\bar{w}_p$  为工序  $p$  指定的工人。
- 节拍波动约束： $\sum_{w \in W} y_w / |W| (1 - volatilityRate) \leq y_w \leq \sum_{w \in W} y_w / |W| (1 + volatilityRate), \forall w \in W$
- 通过求解主问题得到工人和工序的分配关系  $\bar{x}_{wp} \forall w \in W, p \in P$  作为子问题的输入参数。

# 问题分解（自顶向下） 不拆分工序

## 子问题：集合和变量

- 子问题采用 **CP** 建模，建模语言和求解器为 **Google OR-Tools’ CP-SAT**。
- **集合：**
  - $S'$  表示工位集合， $M$  表示设备集合。
  - 通过将原始的工位集合扩展为  $S = S' \cdot \maxCycle$ ，以解决**关键点及难点之“环形生产线的建模”**。扩展模型即包含  $|S|$  个工位的直线生产线。
    - 例如，原始的生产线包含 4 个工位，最大绕圈数为 3，则建模时等价地考虑工位数为  $4 \times 3 = 12$  的直线生产线问题。
  - $S_s$  表示原始工位  $s$  在拓展模型中对应的工位。例如工位  $[1, 2, 3, 4]$  拓展后为  $[1, 2, 3, 4, | 5, 6, 7, 8, | 9, 10, 11, 12]$ ，则  $S_{11} = \{3, 7, 11\}$ 。
- **变量：**
  - 0-1 变量  $x_{ps}$  表示工序  $p$  是否分配给工位  $s \in S$ ， $x_{ps} = 1$  表示是，反之否。（ $S$  为拓展工位序列）
  - 0-1 变量  $x'_{ps}$  表示工序  $p$  是否分配给工位  $s \in S'$ ， $x_{ps} = 1$  表示是，反之否。（ $S'$  为原始工位序列）
  - 0-1 变量  $x_{ws}$  表示工人  $w$  是否分配给工位  $s \in S'$ ， $x_{ws} = 1$  表示是，反之否。
  - 0-1 变量  $x_{ms}$  表示设备  $m$  是否分配给工位  $s \in S'$ ， $x_{ms} = 1$  表示是，反之否。

# 问题分解（自顶向下） 不拆分工序

## 子问题：约束

- 变量链接约束：

- $x_{ps}$  和  $x'_{ps}$  的关系：  $x_{ps'} \Rightarrow x'_{ps}, \forall p \in P, s' \in S_s, s \in S$ ，即若工序  $p$  分配给拓展工位中的  $s' \in S_s$ ，则工序  $p$  必分配给原始工位中的  $s \in S$ 。
- $x_{ps}$  和  $x_{ws}$  的关系：  $x_{ws} = \bar{x}_{wp} \cdot x'_{ps}, \forall w \in W, s \in S', p \in P$ ，其中  $\bar{x}_{wp} \forall w \in W, p \in P$  为主问题传入的参数。

- 工序约束：

- 每个工序仅分配一个工位：  $\sum_{s \in S} x_{ps} = 1, \forall p \in P$
- 工序顺序约束：  $\sum_{s \in S} s \cdot x_{p_1s} \leq \sum_{s \in S} s \cdot x_{p_2s}, \forall (p_1, p_2) \in immediatePrecedence$ ，其中  $immediatePrecedence$  中的工序对具有严格的先后关系。
- 给工序固定工位：  $\sum_{s' \in S_{\bar{s}_p}} x_{ps'} = 1$ ，其中  $\bar{s}_p$  为工序  $p$  的指定工位。即工序  $p$  必须分配给扩展工位中原始工位  $\bar{s}_p$  所对应的工位  $S_{\bar{s}_p}$  之一。

- 工人约束：

- 每个工人至少分配一个工位：  $\sum_{s \in S'} x_{ws} \geq 1, \forall w \in W$ 。
- 每个工人的工位上限：  $\sum_{s \in S'} x_{ws} \leq maxStationPerWorker, \forall w \in W$



# 问题分解（自顶向下） 不拆分工序

## 子问题：约束

- 设备约束：

- 工位需要具备所分配工序所需设备：  $x_{ps} \leq x_{m_p, (s-1)\%|S'|+1}, \forall p \in P, s \in S$ ，其中  $m_p$  为工序  $p$  所需设备， $(s-1)\%|S'|+1$  为拓展工位序列中  $s$  在原始工位序列中的序号。

- 每个工位的最大设备数量：  $\sum_{m \in M'} x_{ms} \leq \maxMachinePerStation$ ，其中  $M' = \{m \in M \mid m_{isMachineNeeded} = True\}$ 。

- 独占工位设备约束：  $x_{m's} \leq 1 - x_{ms}, \forall s \in S', m \in monoMachines, m' \in M - \{m\}$ 。即如某工位有独占设备（ $1 - x_{ms} = 0$ ）则不能再安排其他设备。

- 固定设备约束：  $x_{ms} = 1, \forall s \in S', m \in M_s; x_{ms} = 0, \forall s \in S', m \in M - M_s$ ，其中  $M_s$  为工位  $s$  的固定设备集合。

- 工位约束：

- 每个工位仅分配一个工人：  $\sum_{w \in W} x_{ws} = 1, \forall s \in S'$

# 问题分解（自顶向下） 不拆分工序

## 子问题：重复入站约束

- 引入辅助变量：

- 0-1变量  $z_{sc}$  表示工位  $s \in S'$  在第  $c \in C$  圈是否被使用， $z_{sc} = 1$  表示是，反之否。

- 整数变量  $z_s$  表示工位  $s \in S'$  的重复入站次数。

- 通过以下约束可限制各工位的重复入站次数和总重复入站次数，以解决**关键点及难点**“重复入站约束建模”：

- **表示是否入站：**  $x_{p,s+c|S'|} \Rightarrow z_{sc}, \forall s \in S', c \in C, p \in P$ 。即若存在工序被分配给扩展工位序列中编号为  $s + c|S'|$  的，则位  $s \in S'$  在第  $c \in C$  圈被使用（发生入站）。

- **表示重复入站：**  $z_s = \max\{0, (\sum_{c \in C} z_{cs}) - 1\}, \forall s \in S'$ 。即若发生入站，重复入站次数为总入站数减 1；若未发生入站，重复入站次数为 0。

- **无固定设备工位重复入站次数：**  $z_s \leq 2, \forall s \in S - \bar{S}$ ，其中  $S' - \bar{S}'$  表示无固定设备的工位。

- **总重复入站次数：**  $\sum_{s \in S'} z_s \leq \maxRevisitedStationCount$ 。

- 子问题为可行性问题。若子问题有可行解，则说明主问题已达到全局最优；若子问题无可行解，则生成组合 Benders 割，并将其传递给主问题。



# 问题分解（自顶向下） 不拆分工序

## 组合 Benders 割

- 求解主问题得到工人和工序的分配关系  $\bar{x}_{wp} \forall w \in W, p \in P$  并作为子问题的输入参数，此时子问题可能有两种情况：
  - **子问题可行**：则原始问题的全局最优解（以代理目标函数计）被找到，此时返回主问题得到的工序到工人的分配和子问题得到的工序到工位的分配，即最终的解。
  - **子问题不可行**：则增加如下**组合 Benders 割**到主问题

$$\sum_{\{(w,p)|\bar{x}_{wp}=0\}} x_{wp} + \sum_{\{(w,p)|\bar{x}_{wp}=1\}} (1 - x_{wp}) \geq 1$$

- 该割平面确保本次迭代主问题的解  $\bar{x}_{wp} \forall w \in W, p \in P$  从搜索空间中被删去。因此，算法必然在**有限次迭代内找到最优解**，或证明原问题不可行。

# 算法增强

## 快速不可行性证明和增强组合 Benders 割

- **但是**，该组合 Benders 割及其生成过程有以下因素影响算法整体效率：
  - **该组合 Benders 割较弱**，难以对主问题的搜索方向进行较强的引导（因此这一形式的组合 Benders 割也被称为 no-good cut）。希望尽可能缩小下面两个集合中元素的数量（精准定位导致子问题不可行的原因）。

$$\{(w, p) \mid \bar{x}_{wp} = 0\}, \{(w, p) \mid \bar{x}_{wp} = 1\}$$

- 子问题使用 CP 建模和求解，相比 MIP 能够更加快速的找到可行解（几秒钟内），但在给定的主问题解的条件下**证明子问题不可行仍十分耗时**。

# 算法增强

## 快速不可行性证明和增强组合 Benders 割

- **因此**，设计了启发式算法快速证明子问题不可行。该算法在调用 CP 求解子问题前被调用，很大程度上避免了耗时的 CP 证明子问题不可行。**并且**，该算法可以输出相比  $\{(w,p) | \bar{x}_{wp} = 0\}, \{(w,p) | \bar{x}_{wp} = 1\}$  较小的生成组合 Benders 割的变量集合。
- 算法核心思想为根据工序“**拓扑序**”和工序分配的工人（即主问题的解  $\bar{x}_{wp}$ ），计算所需工位数量的下限。若该下限  $>$  工位数量 + 最大重复入站次数，则子问题必然不可行。且使得该下限  $>$  工位数量 + 最大重复入站次数的**工位集合**可用于增强组合 Benders 割。

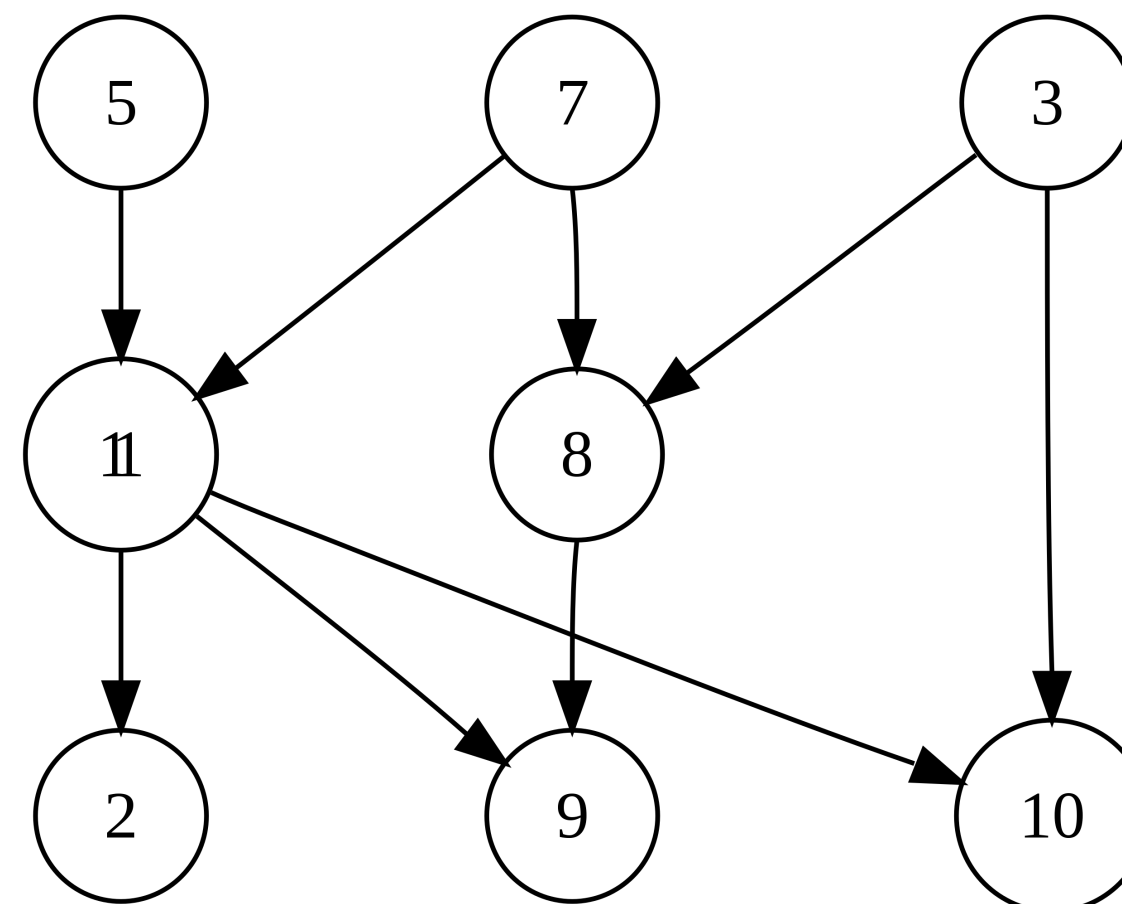
- 右图所示的有向图的拓扑序有：

- 5, 7, 3, 11, 8, 2, 9, 10

- 3, 5, 7, 8, 11, 2, 9, 10

- ...

- 或集合形式的  **$\{3, 5, 7\}, \{8, 11\}, \{2, 9, 10\}$**



有向图的拓扑序

来源: [https://en.wikipedia.org/wiki/Topological\\_sorting](https://en.wikipedia.org/wiki/Topological_sorting)



# 算法增强

## 快速不可行性证明和增强组合 Benders 割

- 详细的实现见 UALB\_CB, Solver, calc\_station\_lb。算法以集合形式的**工序拓扑序**和主问题**工人到工序的分配**关系为输入，输出子问题是否可行，若不可行，同时输出**导致不可行的工序集合**。
- 假设工序拓扑序为  $\{\{1, 2, 3\}, \{4, 5, 6\}, \{7, 8\}, \{9\}, \{10\}, \{11\}, \{12\}\}$ ，表示工序  $\{1, 2, 3\}$  可同时执行，无先后关系要求；但  $\{4, 5, 6\}$  必须在  $\{1, 2, 3\}$  完成后开始执行；其他工序集合间的关系类似。假设以上12个工序被分配给了工人  $[[1, 2, 2], [1, 3, 1], [2, 1], [1], [2], [2], [3]]$ 。下面的例子展示了计算过程：

**工序  $\{1, 2, 3\}$  至少需要 2 个工位（工人 2 做工序 2 和 3，工人 1 做工序 1）；**

**工序  $\{4, 5, 6\}$  至少额外需要 1 个工位（工人 1 做工序 1、4 和 6，工人 3 做工序 5）；**

**工序  $\{7, 8\}$  至少额外需要 1 个工位（工人 1 做工序 4、6 和 8，工人 2 做工序 7）；**

...

- 即通过拓扑序中每个子集中工序和与它**同一子集但排在它之前的工序**分配的工人（例如， $\{1, 2, 3\}$  中 3 之于 1 和 2）；以及子集中工序和它所在子集**前一个子集中**工序分配的工人（例如，4 之于  $\{1, 2, 3\}$ ），确定每个工序子集需要的工位数量。
- 若计算到某一子集所需工位数量  $>$  工位数量 + 最大重复入站次数，则返回“子问题不可行”和该子集之前（包括它在内）的所有工序，记为  $p_{\min}$ 。若把拓扑序倒序排列，同样可得到一组使子问题不可行的工序集合，记为  $p_{\min_r}$ 。

# 算法增强

## 快速不可行性证明和增强组合 Benders 割

- 上述启发式算法对原有基于 CP 的子问题求解有以下提升：
  - 基于规则的启发式算法相比 CP 的不可行性证明，计算负担极大地减轻，从而在相同的运行时间内，子问题能像主问题添加**更多数量的组合 Benders 割**。而组合 Benders 割的数量的多少在一定程度上体现了主问题搜索范围的大小。
  - 并且，所添加的组合 Benders 割

$$\sum_{\{(w,p)|\bar{x}_{wp}=0,w\in W,p\in p_{min}\}} x_{wp} + \sum_{\{(w,p)|\bar{x}_{wp}=1,w\in W,p\in p_{min}\}} (1 - x_{wp}) \geq 1$$
$$\sum_{\{(w,p)|\bar{x}_{wp}=0,w\in W,p\in p_{min\_r}\}} x_{wp} + \sum_{\{(w,p)|\bar{x}_{wp}=1,w\in W,p\in p_{min\_r}\}} (1 - x_{wp}) \geq 1$$

- **相比原始的 no-good cut 更强。**
- 由于快速不可行性证明计算的所需工位数量为下限值，即使该算法不能证明子问题不可行，并不说明子问题一定可行。此时需要调用 CP 求解子问题。根据经验，若子问题可行，则 CP 可以较快速地给出可行解。因此 CP 运行时间超过 CP\_TIME\_LIMIT，则不再运行，直接向主问题添加 Benders 割。

# 算法增强

## 局部分支

- 详细的实现见 `UALB_CB, Solver, __add_local_branching_cut`。
- 赛题属于约束较紧，难以找到可行解的问题。因此引入**局部分支 (Local Branching)**，当主问题找到可行解时搜索该解汉明距离  $k$  之外的解空间，以期尽快找到有效的原问题上界（可行解）。局部分支通过向主问题添加如下的线性不等式实现。

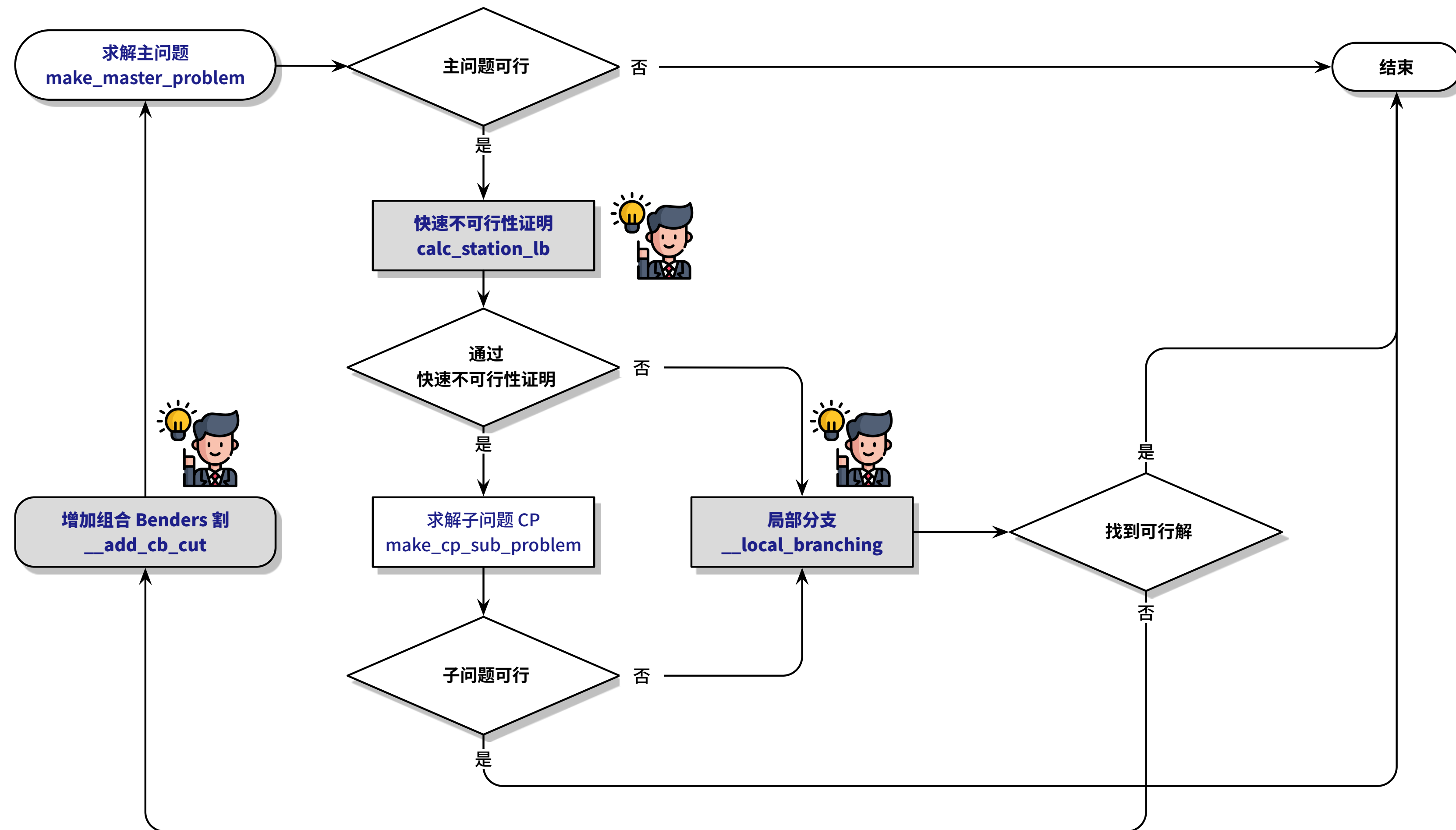
$$\sum_{\{(w,p)|\bar{x}_{wp}=0\}} x_{wp} + \sum_{\{(w,p)|\bar{x}_{wp}=1\}} (1 - x_{wp}) \geq k + 1, \forall k \in LOCAL\_BRANCHING\_K\_SET$$

- 与组合 Benders 割可以保证全局最优解不从解空间中被删除不同，上述局部分支不等式并不能保证这一点。因此局部分支结束时从主问题删除所有已添加的上述不等式。



# 算法流程

## Solver.solve()



# 拆分工序

# 问题分解（自顶向下） 拆分工序

## 主问题目标函数

- 主问题采用 **MIP** 建模，建模语言为 **Pyomo**，求解器为 **HiGHS**。
- **集合：**  $W$  表示工人的集合；  $P$  表示工序的集合
- **变量：**
  - 0-1 变量  $x_{wp}$  表示工人  $w$  是否分配给工序  $p$ ， $x_{wp} = 1$  表示是，反之否。
  - 连续变量  $y_w$  表示工人  $w$  的节拍。
- **工人节拍的计算：**
  - $y_w = \text{standardOperTime}_p / \text{efficiency}_{wp} \cdot (x_{wp} / \sum_{w \in W} x_{wp})$ ，其中  $\text{standardOperTime}_p$  为工序  $p$  的标准工时， $\text{efficiency}_{wp}$  为工人  $w$  做工序  $p$  的效率（可线性化，见源代码 UALB\_CB.py, Solver, make\_master\_problem）。
  - **0 目标函数：Min. 0**（将优化问题转化为可行性问题）
- 最终 **实际参与运算的是“0 目标函数”**，即将原始的优化问题转化为可行性问题。虽然“最小化最大节拍”在不显著增加计算时间的条件下可以实现较小的真实评价指标，但“0 目标函数”在更多的算例上找到了可行解。“最小化基尼系数”会显著增加运算时间。“最小化平均节拍”找到的可行解比“0 目标函数”少，实现的真实评价指标比“最小化最大节拍”大。



# 问题分解（自顶向下） 拆分工序

## 主问题约束

（带 \* 的为与不拆分工序问题有差异的约束）

- **\*每个工序分配给多个工人：**  $1 \leq \sum_{w \in W} x_{wp} \leq \min\{maxWorkerPerOper, maxStationPerOper\}$
- 每个工人至少有一个工序：  $\sum_{p \in P} x_{wp} \geq 1$
- **\*被拆分的工序数量：**  $\sum_{w \in W} x_{wp} \geq 2 \Rightarrow n_p = 1, \sum_{p \in P} n_p \leq MAX\_SPLIT\_TASKS, \forall p \in P$ ，其中  $n_p \in \{0,1\}$  为辅助变量，表示工序是否被拆分（可线性化，见源代码 UALB\_CB.py, Solver, make\_master\_problem）。
- 工序技能和技能类要求：  $\sum_{w \in W} x_{wp} \leq \max(skillCap_{wp}, categoryCap_{wp})$ ，其中  $skillCap_{wp}, categoryCap_{wp} \in \{0,1\}$  表示工人  $w$  是否具有工序  $p$  所需技能或技能类。
- **\*优先分配工序给具有相同技能的工人：**  $\sum_{w \in W_p} x_{wp} \geq 1; \sum_{w \in W - W_p} x_{wp} = 0$ ，其中  $W_p = \{w \in W | skillCapable_{wp} = 1\}$  表示具有工序  $p$  所需技能的工人集合。
- 固定某些工序给特定的工人：  $x_{\bar{w}_p p} = 1, \forall p \in P; x_{wp} = 0, \forall w \in W - \{\bar{w}_p\}$ ，其中  $\bar{w}_p$  为工序  $p$  指定的工人。
- 节拍波动约束：  $\sum_{w \in W} y_w / |W| (1 - volatilityRate) \leq y_w \leq \sum_{w \in W} y_w / |W| (1 + volatilityRate), \forall w \in W$
- 通过求解主问题得到工人和工序的分配关系  $\bar{x}_{wp} \forall w \in W, p \in P$  作为子问题的输入参数。

# 问题分解（自顶向下） 拆分工序

## 子问题的拓展模型

- 子问题结构（集合、参数、和约束）与不拆分工序的情况一致，但需对原有工序集合进行扩展，从而解决**关键点及难点**“**工序拆分的建模**”。例如，原有工序集合为 {1, 2, 3, 4, 5, 6}，其中 {4, 5} 被分别拆分给两个工人。那么，拓展的工序集合为 {1, 2, 3, 4, 5, 6, | 7, 8}，其中 7 具有和 4 完全相同的特征，8 具有和 5 完全相同的特征。具体包括（见 instance.py, Instance, \_make\_dummy\_process）：

由主问题确定的执行该工序的工人；

工序的紧前工序；

工序所需设备；

原始算例数据中工序的所有参数。

- 完成以上操作后，可直接使用不允许拆分工序的子问题求解。并且，设计的快速子问题不可行性证明和局部分支可以兼容以上拓展模型。

# 复杂工序拓扑



# 问题分解（自底向上） 复杂工序拓扑

先确定工序排序，再确定工人分配

- 部分算例**工序拓扑结构复杂（具有较多的拼接工序）**，使用先确定工人分配再确定工序排序的方式难以找到可行解。因此，将主问题和子问题互换，首先确定工序的排序，然后确定工序的工人分配。流程为：
  - 求解**工序的排序问题（主问题）**
  - 将结果带入松弛了节拍波动率的给工序分配工人的问题（松弛子问题）。若可行，则停止；若不可行或可行但不满足节拍波动率约束，则向主问题增加如下**组合 Benders 割**

$$\sum_{\{(p,s)|\bar{x}_{ps}=0\}} x_{ps} + \sum_{\{(p,s)|\bar{x}_{ps}=1\}} (1 - x_{ps}) \geq 1$$

- 该策略在部分自顶向下分解策略无法找到可行解的公开算例上找到了可行解。

# 整体流程

## Solver.run()

- **STEP1:** 尝试**不拆分工序**用**自顶向下**方法求解，若主问题有可行解则说明不需要拆分工序，则不再尝试拆分工序求解；
- **STEP2:** 若STEP1表明无需拆分工序，但在 UALB\_CB\_TIME\_LIMIT 未找到解，则使用**自底向上**方法求解；
- **STEP3:** 若STEP1表明不拆分工序无可行解则尝试**拆分工序**用**自顶向下**方法求解；
- **STEP4:** 若在规定时间内找到解则将解输入 Solution 并保存求解结果。

# 数值试验结果

初赛成绩：3.0018（排名：3）  
公开算例表现：

|          | 数量 | 解数量<br>(0 / 最大节拍) | 平均求解时间<br>(Min. 0) | 真实评价指标<br>(Min. 0) | 平均求解时间<br>(Min. 最大节拍) | 真实评价指标<br>(Min. 最大节拍) |
|----------|----|-------------------|--------------------|--------------------|-----------------------|-----------------------|
| 不拆分 / 简单 | 29 | 29 / 29           | 5.72 s             | 0.3668             | 7.83 s                | 0.2911                |
| 不拆分 / 困难 | 16 | 10 / 8            | 195.75 s           | 3.9853             | 224.62                | 5.1879                |
| 拆分 / 简单  | 10 | 10 / 10           | 8.87 s             | 0.3703             | 6.58 s                | 0.2507                |
| 拆分 / 困难  | 2  | 0 / 0             | /                  | /                  | /                     | /                     |

注：另有 3 个疑似不可行算例。程序运行设备硬件：4C4G



# 算法优势

|       | 基于 Benders 分解的<br>精确算法 | 基于规则的<br>启发式算法 | 引入随机的<br>元启发式算法 |
|-------|------------------------|----------------|-----------------|
| 解的最优性 | 理论保证全局最优               | 无法保证           | 无法保证            |
| 解的可行性 | 可以保证                   | 取决于具体算法设计      | 取决于具体算法设计       |
| 速度    | 2/3以上公开算例秒级给出最优解或可行解   | 快              | 取决于具体算法设计       |
| 拓展性   | 易增加或减少约束               | 取决于具体约束类型      | 取决于具体约束类型       |

# 结论和展望

- 结论：

- 针对环形生产线平衡问题，基于 **CBD 和 LBB** 开发了求解算法。将主问题建模为MIP，子问题建模为 CP。实现了**快速证明子问题不可行的启发式算法**，并使用**局部分支**增强算法搜索可行解的能力。
- **自顶向下**的分解策略可适用于具有简单拓扑的任务不可拆分问题及任务可拆分问题；**自底向上**的分解策略使用于具有复杂拓扑的任务不可拆分问题。算法具有较强的扩展性。

- 展望：

- 使用 Branch-and-Benders-Decomposition 在单个主问题分支定界树（branch-and-bound tree）上进行搜索，进一步提升算法效率。
- 将自底向上的分解策略用于具有复杂拓扑的任务可拆分问题。
- 增强自底向上的分解策略的组合 Benders 割。
- 拆分工序条件下的全局最优解。

# 参考文献

## ALBP & Benders 分解 & 局部分支 & 代理目标函数

1. Boysen, N., Schulze, P., & Scholl, A. (2022). Assembly line balancing: What happened in the last fifteen years? *European Journal of Operational Research*, 301(3), 797–814. <https://doi.org/10.1016/j.ejor.2021.11.043>
2. Benders, J. F. (1962). Partitioning procedures for solving mixed-variables programming problems. *Numerische Mathematik*, 4(1), 238–252. <https://doi.org/10.1007/BF01386316>
3. Codato, G., & Fischetti, M. (2006). Combinatorial Benders' cuts for mixed-integer linear programming. *Operations Research*, 54(4), 756–766. <https://doi.org/10.1287/opre.1060.0286>
4. Hooker, J. N., & Ottosson, G. (2003). Logic-based Benders decomposition. *Mathematical Programming*, 96(1), 33–60. <https://doi.org/10.1007/s10107-003-0375-9>
5. Rahmaniani, R., Crainic, T. G., Gendreau, M., & Rei, W. (2017). The Benders decomposition algorithm: A literature review. *European Journal of Operational Research*, 259(3), 801–817. <https://doi.org/10.1016/j.ejor.2016.12.005>
6. Fischetti, M., & Lodi, A. (2003). Local branching. *Mathematical Programming*, 98(1), 23–47. <https://doi.org/10.1007/s10107-003-0395-5>
7. Rei, W., Cordeau, J.-F., Gendreau, M., & Soriano, P. (2009). Accelerating Benders Decomposition by Local Branching. *INFORMS Journal on Computing*, 21(2), 333–345. <https://doi.org/10.1287/ijoc.1080.0296>
8. Tsang, M. Y., & Shehadeh, K. S. (2022). Convex Fairness Measures: Theory and Optimization (arXiv:2211.13427). arXiv. <http://arxiv.org/abs/2211.13427>



# 谢谢！

郭鹏辉 博士生（Min 团队）

<https://guo.ph>

南京航空航天大学 经济与管理学院

智能决策与风险分析研究所